



PEC3 - Tercera prueba de evaluación continuada

Presentación

Esta PEC se focaliza en los circuitos secuenciales. Los circuitos combinacionales nos permiten describir funcionalidades de un circuito pero no nos permite guardar información. Mediante biestables y registros podemos guardar información en memoria y hacer circuitos más complejos. En este PEC practicaremos con este tipo de circuitos.

Competencias

- Entender el funcionamiento de los circuitos lógicos secuenciales y conocer y saber aplicar técnicas de diseño de sistemas secuenciales.

Objetivos

- Saber discernir, a partir de la funcionalidad que se quiere que tenga un circuito lógico, si el circuito tiene que ser de tipo secuencial o combinacional.
- Conocer el funcionamiento del biestable D y de todas las entradas de control que puede tener.
- Saber analizar un circuito secuencial.
- Saber realizar un cronograma a partir de un circuito digital secuencial.
- Saber analizar un grafo de estados.
- Saber diseñar un circuito cualquiera a partir de la descripción de su funcionalidad mediante el modelo de Moore.

Recursos

Los recursos que se recomienda usar para esta PEC son los siguientes:

Básicos: El módulo 4 de los materiales.

Complementarios: VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.

Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.

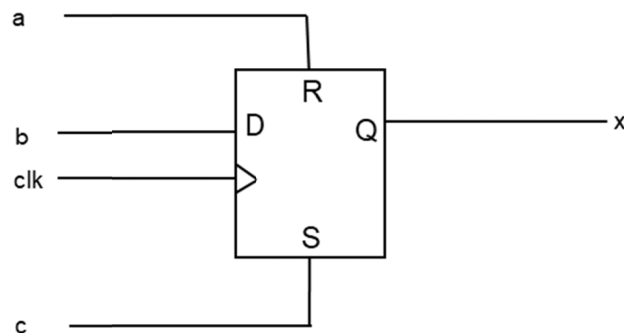


- Hay que entregar la solución en un fichero PDF usando una de las plantillas publicadas conjuntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **27 de abril** (a las 24 horas).

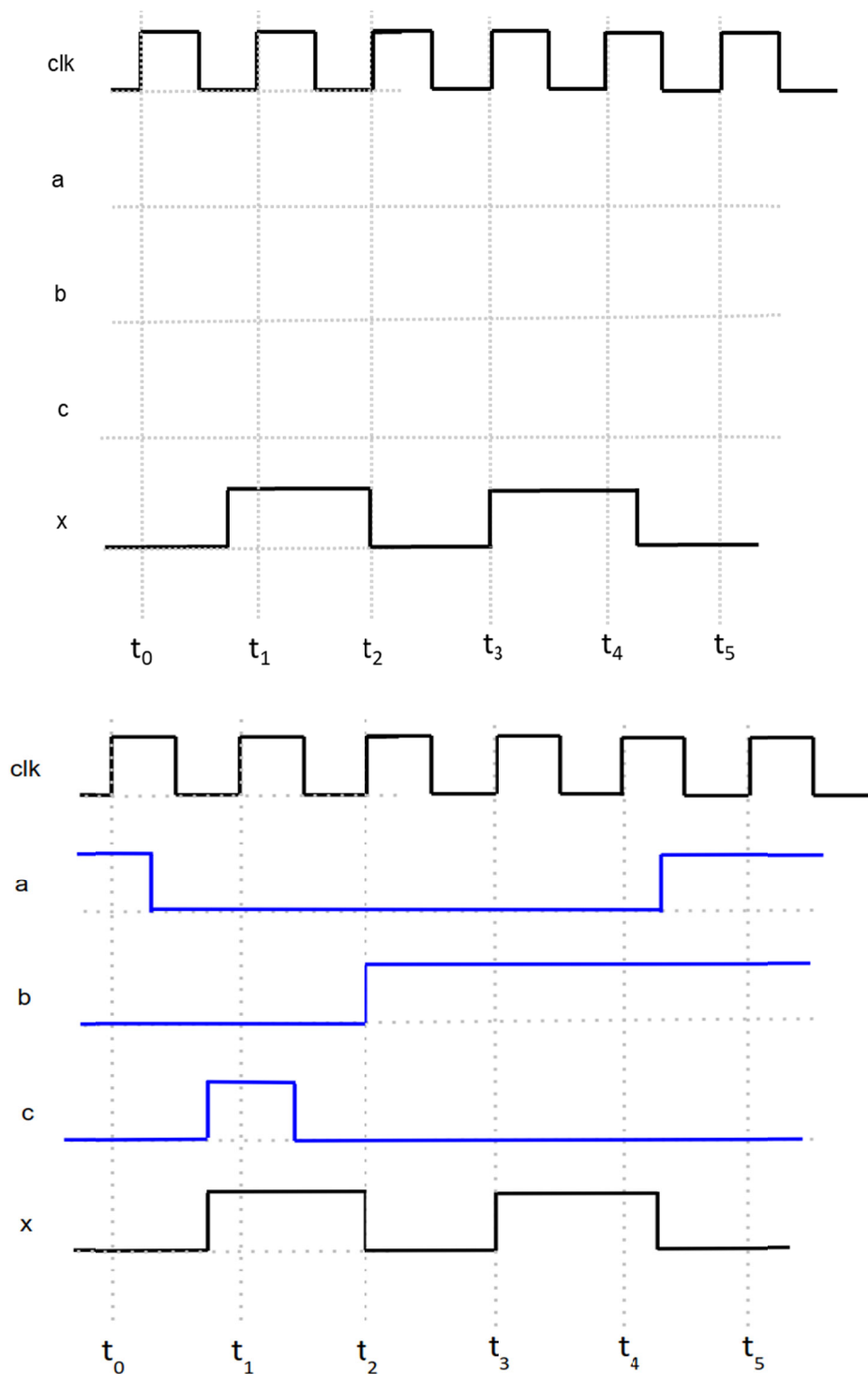
Solución de la PEC

Ejercicio 1 [10%]

Dado el circuito que se muestra a continuación:



Completa el siguiente cronograma con unos posibles valores para las señales de entrada a , b y c , tales que hacen compatible que se obtengan los valores de la salida x . Además, indicad **razonadamente** los valores iniciales que habéis puesto y todos los cambios en los valores de las señales de entrada.



Hasta t_0 : La señal asíncrona $a = 1$ ha puesto el valor de la salida x a cero.

En el intervalo t_0 - t_1 : Se activa la señal asíncrona $c = 1$ para poner la señal x al valor 1 de forma inmediata.



t_2 : La señal x toma el valor 0 de la entrada $b = 0$. Las señales asíncronas a y c tienen el valor cero.

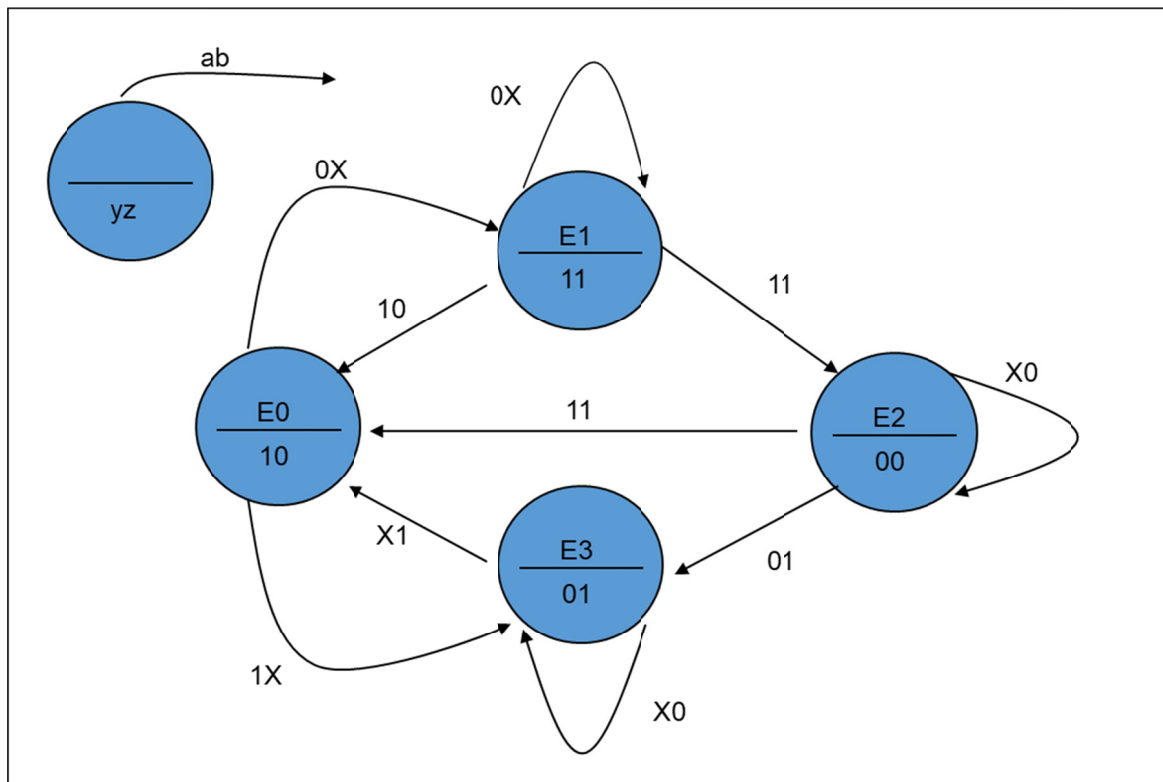
t_3 : La señal x toma el valor 1 de la entrada $b = 1$. Las señales asíncronas a y c tienen el valor cero. La señal b se tiene que poner a 1 en cualquier momento entre t_2 y t_3 .

t_4 : La señal x toma el valor 1 de la entrada $b = 1$. Las señales asíncronas a y c tienen el valor cero.

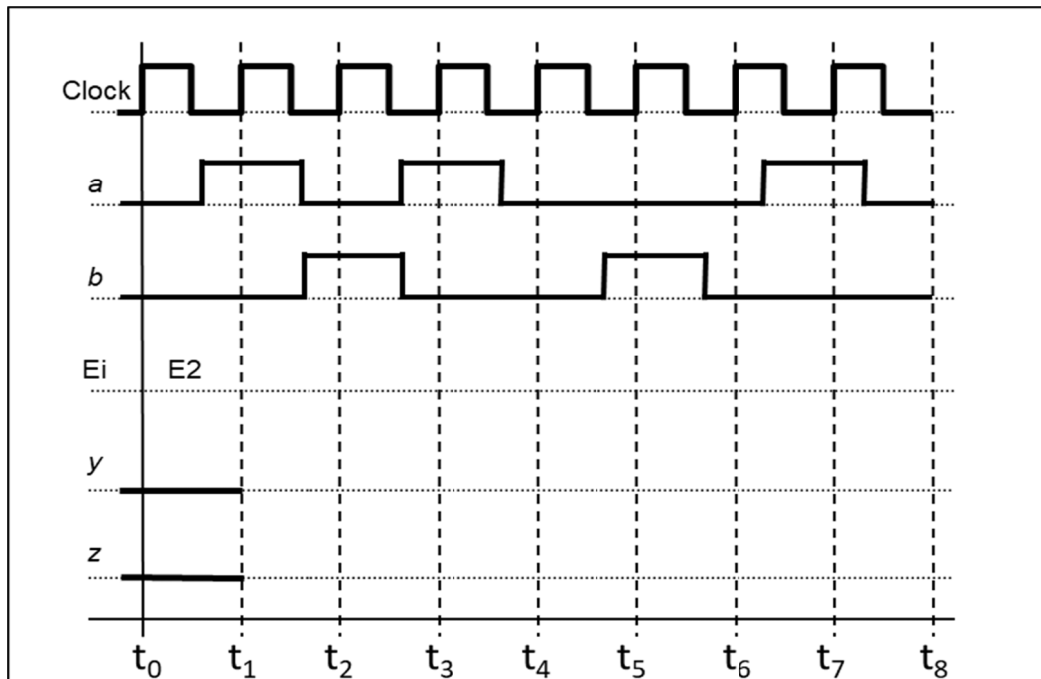
En el intervalo t_4 - t_5 : Se activa la señal asíncrona $a = 1$ para poner la señal x al valor 0 de forma inmediata. La señal a se mantiene al valor 1 hasta el final del cronograma. Durante este ciclo la señal b podría haber bajado a 0 en cualquier momento y también sería coherente con la salida dada.

Ejercicio 2 [15%]

Dado este grafo de estados:

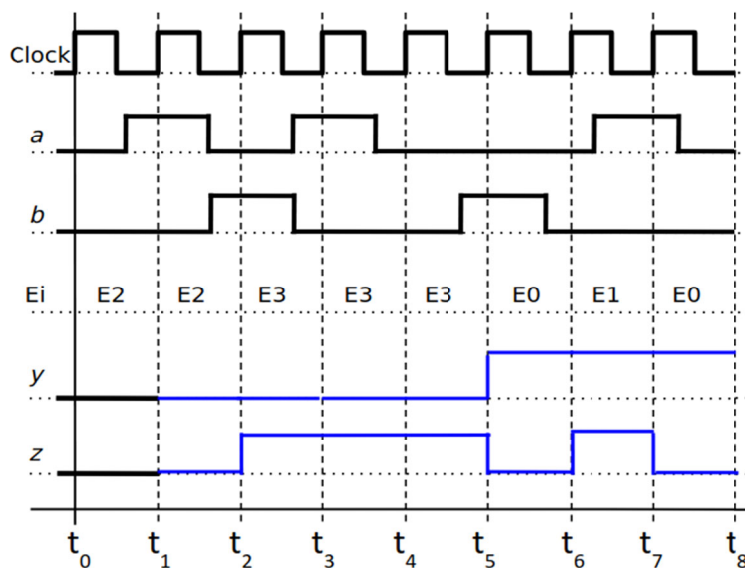


Completa el siguiente cronograma.



Nota: Tenéis disponible este ejercicio en VerilCHART.

Para determinar el estado futuro miramos en los flancos ascendentes de la señal de reloj los valores que hay a las variables de entrada justo antes del flanco. En función del estado actual y de estos valores determinamos, según indique el grafo de estados dado, cuál tiene que ser el nuevo estado. Las salidas se determinan en función del estado, y el valor que tienen que tener estos bits los determina el grafo de estados dado. El cronograma resultante es:





Ejercicio 3 [25%]

Dado el grafo de estados del ejercicio anterior, implementadlo usando una única memoria ROM y biestables.

Indicad claramente las conexiones del circuito que realicéis, teniendo en cuenta que los bits de más peso de cada palabra de la ROM tienen que establecer la transición entre estados que hace falta y los bits de menos peso establecen la salida correspondiente.

Tenéis que indicar el contenido (en binario) de todas las direcciones de la ROM. La codificación de los estados se tiene que hacer según su índice asociado.

NOTA: Tenéis disponible el ejercicio en VerilCIRC.

Codificamos en la siguiente tabla los estados según su índice:

Codificación de los estados	
Estado	q_1q_0
$E0$	00
$E1$	01
$E2$	10
$E3$	11

La tabla de transiciones establece cuál es el estado en el ciclo siguiente a partir del valor que toman las señales de entrada.

q_1q_0	ab	q_1+q_0+
00	00	01
00	01	01
00	10	11
00	11	11
01	00	01
01	01	01
01	10	00
01	11	10
10	00	10
10	01	11
10	10	10
10	11	00
11	00	11
11	01	00
11	10	11
11	11	00

La tabla de salidas determina el valor de las señales de salida para cada estado.

Estado	yz
00	10



01	11
10	00
11	01

Finalmente, unificamos las dos tablas en una, teniendo en cuenta que las salidas yz dependen del estado, no del estado+. A continuación, se muestra la tabla unificada:

q_1q_0	ab	q_1+q_0+	yz
00	00	01	10
00	01	01	10
00	10	11	10
00	11	11	10
01	00	01	11
01	01	01	11
01	10	00	11
01	11	10	11
10	00	10	00
10	01	11	00
10	10	10	00
10	11	00	00
11	00	11	01
11	01	00	01
11	10	11	01
11	11	00	01

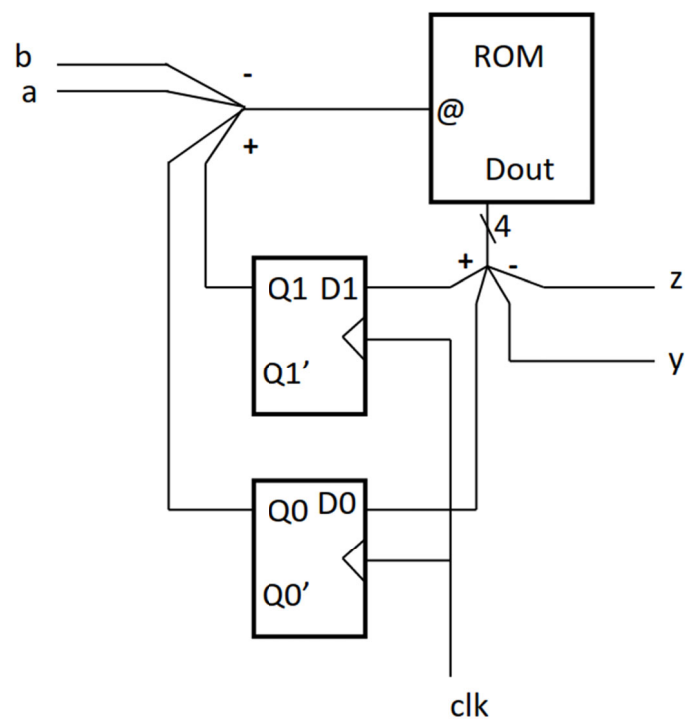
El direccionamiento de la memoria ROM viene definido por las señales $q_1 q_0$ a b especificadas en orden de mayor a menor peso. Cada posición de la memoria almacena los bits $q_1+ q_0+ y z$ especificados en orden de mayor a menor peso. El contenido de la memoria ROM es:

Dirección	Contenido en binario
0	0110
1	0110
2	1110
3	1110
4	0111
5	0111
6	0011
7	1011
8	1000
9	1100



10	1000
11	0000
12	1101
13	0001
14	1101
15	0001

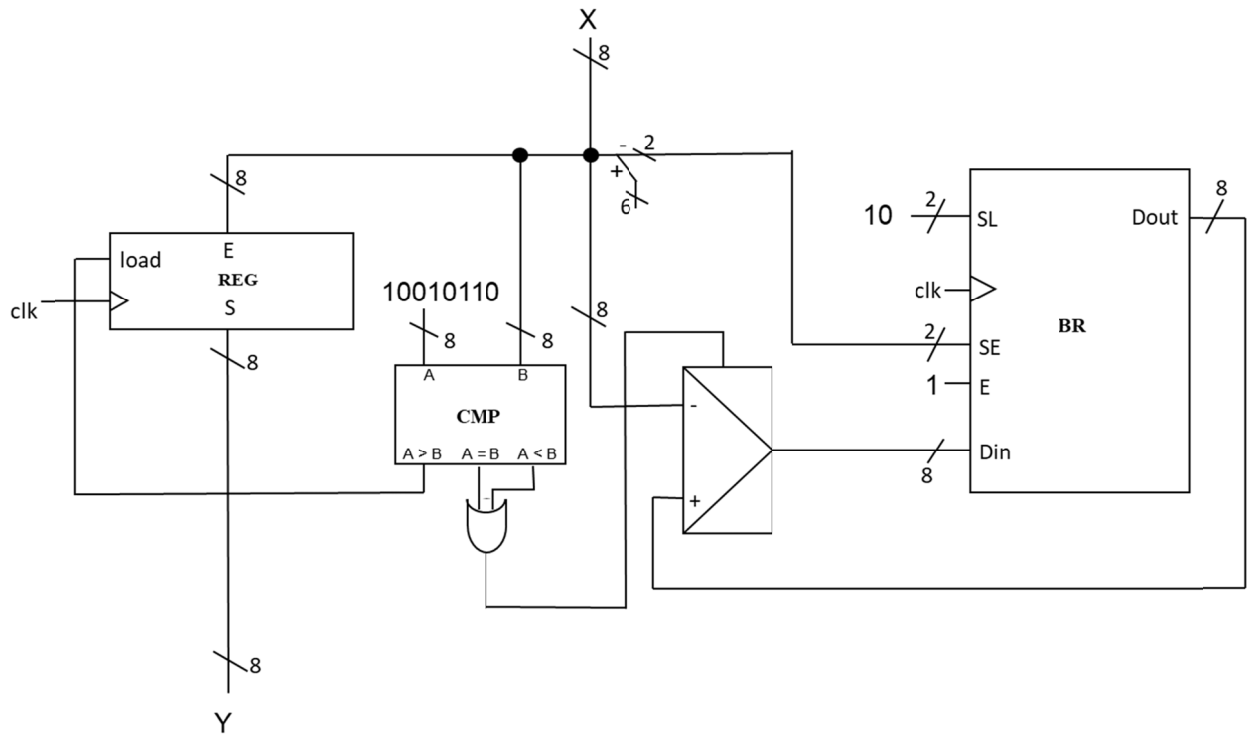
A continuación, se muestra el circuito correspondiente.





Ejercicio 4 [30%]

Dado el siguiente circuito, donde BR es un banco de 4 registros que denominamos R0, R1, R2 y R3.



a) [10%] Sabiendo que en el momento anterior de producirse un flanco ascendente de reloj los valores de los registros son (en binario):

Y = 00001001

R0 = 00000011

R1 = 00001100

R2 = 00110000

R3 = 11000000

y el valor de la entrada es (en binario):

X = 00001111

Indicad **razonadamente** cuáles serán los valores que contendrán los registros (Y, R0, R1, R2, R3) justo en el momento posterior de este flanco ascendente.

Con la entrada X = 00001111 el valor de la entrada SE es 11, por lo tanto, se escribirá en el registro R3. Por lo tanto, los valores de los registros R0, R1 y R2 no cambian y son: **R0 = 00000011, R1 = 00001100, R2 = 00110000.**

La entrada A=10010110 del comparador es más grande que la entrada B=00001111. Por lo tanto, la salida de la puerta OR tiene el valor 0. Con el valor 0 en la entrada de control del multiplexor se selecciona la entrada de datos de menor peso. Por lo tanto, el valor de la señal X se escribirá vía Din del BR al registro R3. **R3=00001111.**



Como la salida del comparador $A > B = 1$, la entrada load del registro tiene el valor 1 y se escribirá el valor de la entrada X al registro. Por lo tanto, $Y = 00001111$.

b) [10%] Sabiendo que en el momento posterior de producirse un flanco ascendente de reloj los valores de los registros son (en binario):

Y = 00001001
R0 = 00000011
R1 = 00110000
R2 = 00110000
R3 = 11000000

Indicad **razonadamente** todos los valores que podría tener la entrada X justo en el momento anterior del flanco de forma que los valores posteriores sean coherentes. Si no puede haber ninguno, indicadlo justificándolo.

Según el valor de la entrada de control del multiplexor, se escribe el valor de X o el valor del registro R2 (puesto que SL selecciona el registro R2) al BR. El registro destino concreto del BR está determinado por los 2 bits de menor peso de la entrada X.

El valor de $Y = 00001001$ no puede haber sido el valor de X porque con $A = 10010110$ la salida $A < B$ y $A = B$ es cero y se hubiera escrito el valor de X al registro 1 ($SE = 01$).

Dado que no se había activado la entrada load del registro, la entrada X tiene que haber sido mayor o igual al valor de A.

Con un valor de X mayor o igual a A, la entrada de control del multiplexor es 1 y por tanto el multiplexor selecciona como entrada el valor de Dout, que es el valor del registro R2 (puesto que $SL = 10$).

Los dos registros R2 y R1 que tienen el mismo valor (que corresponde al valor de R2 para la entrada Din). Pues los últimos dos bits de menor peso de X que determinan el valor de SE tendrán que haber sido 01 o 10.

En resumen, la entrada X tiene que haber sido mayor o igual a 10010110 y los dos bits de menor peso de X tienen que haber tenido el valor 01 o 10.

c) [10%] Sabiendo que en el momento posterior de producirse un flanco ascendente de reloj los valores de los registros son (en binario):

Y = 01010101
R0 = 00000011
R1 = 01100110
R2 = 10011001
R3 = 11000011

Indicad **razonadamente** todos los valores que podría tener la entrada X justo en el momento anterior del flanco de forma que los valores posteriores sean coherentes. Si no puede haber ninguno, indicadlo justificándolo.



El caso de este apartado es similar al del anterior apartado.

Si $X < 10010110_2$ entonces X se hubiera escrito en el registro Y y también en algún registro del BR. Debido a que no hay coincidencia del valor de Y con ningún valor de BR este caso no se ha podido dar.

Si $X \geq 10010110_2$ entonces no se ha escrito en Y pero sí que ha habido una escritura en BR. El único caso que se puede dar es que un registro se copie a otro ya que la entrada de control del MUX así lo selecciona. Ya que no hay ningún registro con un valor coincidente significa que la única posibilidad es que se haya sobreescrito el mismo registro, siendo ésta la diferencia respecto al apartado anterior. Como que el registro que siempre se lee es el $10_2 = 2$ significa que los dos últimos bits de X deben ser 10_2 .

Por tanto, X debe haber tenido un valor mayor o igual que 10010110_2 y los dos últimos bits tenían que ser 10_2 .

Ejercicio 5 [20%]

Diseñad el grafo de estados de un circuito que detecte si la secuencia de datos en la entrada es capicúa.

En concreto el circuito tiene una entrada denominada e por donde van llegando los datos y una salida c donde indica si se ha detectado el capicua (1) o no (0). La condición de capicúa se establece tal que los 5 valores que haya en la entrada durante los ciclos posteriores a la inicialización sean simétricos. Por ejemplo, la secuencia de valores 1 0 0 0 1 generaría un 1 en la salida mientras que a la secuencia 0 1 1 0 1 le corresponde un 0 en la salida. La salida se tiene que mantener estable hasta que el circuito se vuelva a inicializar.

Indicad **razonadamente** la funcionalidad de cada estado intentando minimizar el número de estados que es necesario definir.



Para conseguir el funcionamiento deseado, el circuito tiene que tener los siguientes estados:

Estado	Descripción	Salida
INI	Estado inicial, no se ha reconocido nada de la secuencia	0
E0	Se ha visto un 0	0
E1	Se ha visto un 1	0
E2	Se ha visto la secuencia 11	0
E3	Se ha visto la secuencia 10	0
E4	Se ha visto la secuencia 01	0
E5	Se ha visto la secuencia 00	0
E6	Se ha visto la secuencia 11x	0
E7	Se ha visto la secuencia 10x	0
E8	Se ha visto la secuencia 01x	0
E9	Se ha visto la secuencia 00x	0
E10	Se ha visto la secuencia 11x1 o 10x0	0
E11	Se ha visto una secuencia no simétrica	0
E12	Se ha visto la secuencia 01x1 o 00x0	0
E13	Se ha visto una secuencia simétrica	1

El siguiente grafo representa el comportamiento deseado:

